OpenCV Python Course Notes

English + Roman Urdu | Beginner Friendly | With Code Examples

Module	Topic
1	Getting Started with Images
2	Basic Image Manipulation
3	Image Annotation
4	Image Enhancement
5	Accessing the Camera
6	Read and Write Videos
7	Image Filtering and Edge Detection
8	Image Features and Image Alignment

Install: `pip install opencv-python matplotlib numpy`
Import: `import cv2`

> Note: Yeh notes course ke 8 modules ke standard topics par based hain. Code run karke khud practice zaroor karein.

Module 1: Getting Started with Images

1.1 What is a digital image?

EN: A digital image is a grid (matrix) of tiny dots called **pixels**. Each pixel stores a number. In a grayscale image the number is 0 (black) to 255 (white). A colour image has 3 numbers per pixel: Blue, Green, Red.

> **Roman Urdu:** Digital image ek jaali (grid) hoti hai jis mein chhote chhote dots hotay hain jinhein **pixels** kehte hain. Har pixel mein ek number hota hai. Grayscale mein 0 matlab kala aur 255 matlab safed. Colour image mein har pixel ke 3 numbers hotay hain: Blue, Green, Red.

Example (Roman Urdu): Socho ek chai ki cup ki photo hai. Photo ke har chhote point ka ek number hai. Computer photo nahi dekhta, sirf numbers ka table dekhta hai.

EN: Important: OpenCV reads colour in **BGR** order, not RGB. Matplotlib uses RGB. So you must convert before showing with matplotlib.

> **Roman Urdu: Zaroori:** OpenCV colour ko **BGR** order mein parhta hai, RGB mein nahi. Matplotlib RGB use karta hai, is liye matplotlib mein dikhane se pehle convert karna parta hai.

1.2 Read, display and save an image

```
import cv2
img = cv2.imread("cup.jpg")            # read (BGR)
gray = cv2.imread("cup.jpg", 0)        # 0 = read as grayscale
print(img.shape)                       # (height, width, channels)
print(img.dtype)                       # uint8

cv2.imshow("Window", img)              # show
cv2.waitKey(0)                         # wait for key press
cv2.destroyAllWindows()                # close windows

cv2.imwrite("saved.png", img)          # save
```

> **Roman Urdu: imread** image ko memory mein load karta hai. **imshow** window mein dikhata hai. **waitKey(0)** keyboard ka intezar karta hai warna window foran band ho jati hai. **imwrite** file save karta hai.

Example (Roman Urdu): Agar aap waitKey(0) na likhein to window ek second ke liye khulegi aur band ho jayegi, jaise koi mehmaan aaye aur foran chala jaye.

Show image with Matplotlib

```
import matplotlib.pyplot as plt
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb)
plt.axis("off")
plt.show()
```

> **Roman Urdu:** Jupyter Notebook mein cv2.imshow ki jagah matplotlib use karte hain. Pehle **cvtColor** se BGR ko RGB mein badalna zaroori hai warna rang ulta dikhega (neela chehra!).

1.3 Split, merge and colour spaces

EN: An image can be split into its channels (B, G, R) and merged back. A **colour space** is another way of describing colour. Common ones: BGR, RGB, GRAY, HSV.

> **Roman Urdu:** Image ko uske channels (B, G, R) mein torr sakte hain aur wapis jor sakte hain. **Colour space** rang ko likhne ka alag tareeqa hai. Aam colour spaces: BGR, RGB, GRAY, HSV.

```
b, g, r = cv2.split(img)
merged = cv2.merge((b, g, r))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv  = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

```
h, s, v = cv2.split(hsv)     # Hue, Saturation, Value
```

Example (Roman Urdu): HSV mein **Hue** = rang kaun sa hai (laal, hara), **Saturation** = rang kitna gehra hai, **Value** = kitna roshan hai. Agar aapko sirf laal gend dhoondni hai to HSV mein dhoondna RGB se asaan hai.

Module 2: Basic Image Manipulation

EN: Images are NumPy arrays, so we can crop, resize and flip them using simple array operations or OpenCV functions.

> **Roman Urdu:** Images NumPy arrays hoti hain, is liye hum array operations ya OpenCV functions se crop, resize aur flip kar sakte hain.

2.1 Cropping

```
# img[y1:y2, x1:x2]   (rows first, then columns!)
crop = img[100:300, 200:400]
```

> **Roman Urdu:** Crop karte waqt pehle **y (rows)** aur phir **x (columns)** likhte hain. Yeh aksar students ghalat kar dete hain.

Example (Roman Urdu): Jaise class ki seating: pehle ye batao kaun si line (row), phir kaun si seat (column).

2.2 Resizing

```
# by scale factor
small = cv2.resize(img, None, fx=0.5, fy=0.5)

# by exact size (width, height)
fixed = cv2.resize(img, (300, 200), interpolation=cv2.INTER_AREA)
```

EN: Caution: in cv2.resize size is (width, height), but img.shape gives (height, width). Use INTER_AREA when shrinking and INTER_LINEAR or INTER_CUBIC when enlarging.

> **Roman Urdu: Khayal rakhein:** resize mein size **(width, height)** hota hai jabke img.shape **(height, width)** deta hai. Chhota karna ho to INTER_AREA, bara karna ho to INTER_LINEAR ya INTER_CUBIC use karein.

Example (Roman Urdu): 500x500 ki photo ko WhatsApp DP ke liye 100x100 banana ho to resize use hoga.

2.3 Flipping

```
flip_v  = cv2.flip(img, 0)    # vertical  (upside down)
flip_h  = cv2.flip(img, 1)    # horizontal (mirror)
flip_hv = cv2.flip(img, -1)   # both
```

> **Roman Urdu: 0** = ulta (upar neeche), **1** = aaine jaisa (left right), **-1** = dono.

Example (Roman Urdu): Selfie camera mirror image dikhata hai, yeh flip(img, 1) jaisa hi hai.

Module 3: Image Annotation

EN: Annotation means drawing shapes and writing text on an image. It is used to mark detected objects, such as drawing a box around a face. All drawing functions change the image **in place**, so work on a copy if you want to keep the original.

> **Roman Urdu:** Annotation ka matlab image par shapes banana aur text likhna hai. Yeh detect kiye gaye objects ko mark karne ke liye hota hai, jaise chehre ke gird box. Drawing functions original image ko **direct badal dete hain**, is liye original bachana ho to pehle **copy()** bana lein.

```
out = img.copy()

# Line:      (image, start, end, color BGR, thickness)
cv2.line(out, (50, 50), (300, 50), (0, 0, 255), 3)

# Circle:    (image, center, radius, color, thickness)
cv2.circle(out, (200, 200), 60, (0, 255, 0), 2)

# Rectangle: (image, top-left, bottom-right, color, thickness)
cv2.rectangle(out, (100, 100), (300, 250), (255, 0, 0), 2)

# Text:      (image, text, origin, font, scale, color, thickness, lineType)
cv2.putText(out, "Hello OpenCV", (50, 320),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA)
```

> **Roman Urdu:** Colour **BGR** mein dete hain: (0,0,255) = laal, (0,255,0) = hara, (255,0,0) = neela. **thickness = -1** likhein to shape andar se bhar jati hai. Point hamesha **(x, y)** hota hai.

Example (Roman Urdu): Maan lo aapne cricket match ki photo mein ball ke gird gola lagana hai: `cv2.circle(out, (ball_x, ball_y), 20, (0,0,255), 2)` likhenge aur ball laal gole mein nazar aayegi.

EN: Tip: thickness=-1 gives a filled shape. Use cv2.LINE_AA for smooth (anti-aliased) edges.

Module 4: Image Enhancement

EN: Enhancement improves the look of an image or prepares it for analysis. Topics: brightness, contrast, thresholding, bitwise operations.

> **Roman Urdu:** Enhancement image ki quality behtar karta hai ya usay analysis ke liye tayyar karta hai. Topics: brightness, contrast, thresholding, bitwise operations.

4.1 Brightness (addition)

```
import numpy as np
matrix = np.ones(img.shape, dtype="uint8") * 50
brighter = cv2.add(img, matrix)
darker   = cv2.subtract(img, matrix)
```

> **Roman Urdu:** Har pixel mein 50 jama karne se image roshan hoti hai, minus karne se andheri. **cv2.add** 255 se upar nahi jata (saturate ho jata hai), jabke NumPy ka **+** 255 ke baad wapis 0 se shuru kar deta hai aur ajeeb dhabbe aa jate hain.

Example (Roman Urdu): Raat ki dhundli photo ko roshan karna ho to har pixel mein thora sa brightness add kar do.

4.2 Contrast (multiplication)

```
low  = np.ones(img.shape, dtype="float64") * 0.8
high = np.ones(img.shape, dtype="float64") * 1.2
img_low  = np.uint8(np.clip(cv2.multiply(np.float64(img), low),  0, 255))
img_high = np.uint8(np.clip(cv2.multiply(np.float64(img), high), 0, 255))
```

> **Roman Urdu:** Contrast ka matlab roshan aur andheray hisson ka farq. Pixels ko 1 se zyada se multiply karein to contrast barhta hai, 1 se kam se multiply karein to ghat jata hai. **np.clip** values ko 0 se 255 ke andar rakhta hai.

4.3 Thresholding

EN: Thresholding converts a grayscale image into a pure black-and-white (binary) image. Pixels above the threshold become white, others become black.

> **Roman Urdu:** Thresholding grayscale image ko sirf kala aur safed (binary) image bana deti hai. Threshold se bare pixels safed, baqi kale.

```
gray = cv2.imread("book.jpg", 0)

# global threshold
ret, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# adaptive threshold (good when lighting is uneven)
adaptive = cv2.adaptiveThreshold(gray, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 7)
```

Example (Roman Urdu): Kitaab ke page ko scan karte waqt ek taraf saya ho to global threshold fail ho jata hai, adaptive threshold har chhote hisse ke liye alag threshold nikalta hai is liye behtar chalta hai.

4.4 Bitwise operations

```
and_img = cv2.bitwise_and(img1, img2, mask=None)
or_img  = cv2.bitwise_or(img1, img2)
xor_img = cv2.bitwise_xor(img1, img2)
not_img = cv2.bitwise_not(img1)
```

> **Roman Urdu:** Bitwise operations masks ke saath use hote hain: image ka sirf wo hissa rakhna jo mask mein safed hai. Logo lagana ya background hatana inhi se hota hai.

Example (Roman Urdu): Aapke paas ek gol mask hai. `bitwise_and(img, img, mask=mask)` sirf gole ke andar ki photo rakhega aur baqi kala kar dega.

Module 5: Accessing the Camera

EN: cv2.VideoCapture(0) opens the default webcam (0 = first camera, 1 = second, etc.). We read frames in a loop, because video is just many images shown quickly.

Roman Urdu: cv2.VideoCapture(0) default webcam kholta hai (0 = pehla camera, 1 = doosra). Hum loop mein frames parhte hain kyunke video bohat si images ka tez silsila hota hai.

```
import cv2
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Camera not opened")
    raise SystemExit

while True:
    ret, frame = cap.read()          # ret = True if frame read OK
    if not ret:
        break
    cv2.imshow("Webcam", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):   # press q to quit
        break

cap.release()
cv2.destroyAllWindows()
```

Roman Urdu: cap.read() do cheezein deta hai: **ret** (kya frame mil gaya?) aur **frame** (asal image). **waitKey(1)** 1 millisecond rukta hai aur key check karta hai. Aakhir mein **cap.release()** zaroor likhein warna camera dusri app ke liye locked reh sakta hai.

Example (Roman Urdu): Yeh bilkul TV ki tarah hai: har second mein 30 tasveerein tezi se badalti hain aur hamein chalti hui video nazar aati hai.

EN: Camera properties: cap.get(cv2.CAP_PROP_FRAME_WIDTH) and cap.get(cv2.CAP_PROP_FPS) return width and frame rate.

Roman Urdu: Camera ki width, height aur FPS (frames per second) **cap.get()** se maloom hoti hai.

Module 6: Read and Write Videos

6.1 Reading a video file

EN: Reading a video file is the same as reading the webcam, only the argument is a file path.

> **Roman Urdu:** Video file parhna webcam jaisa hi hai, bas 0 ki jagah file ka path dete hain.

```
cap = cv2.VideoCapture("race_car.mp4")
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:                 # video ended
        break
    cv2.imshow("Video", frame)
    if cv2.waitKey(25) & 0xFF == 27:   # 27 = ESC key
        break
cap.release()
cv2.destroyAllWindows()
```

> **Roman Urdu: waitKey(25)** ka matlab har frame ke baad 25 ms ruko, yani lagbhag 40 FPS ki speed. Zyada number dein to video slow ho jati hai.

6.2 Writing a video file

EN: We need a **VideoWriter** with a codec (FourCC), FPS and frame size. The frame size must be (width, height) and must exactly match the frames you write.

> **Roman Urdu: VideoWriter** ko codec (FourCC), FPS aur frame size chahiye. Size **(width, height)** hona chahiye aur frames ki size se bilkul match karna chahiye warna video save nahi hoti.

```
cap = cv2.VideoCapture(0)
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*"XVID")
out = cv2.VideoWriter("output.avi", fourcc, 20.0, (w, h))

while True:
    ret, frame = cap.read()
    if not ret:
        break
    out.write(frame)                # save this frame
    cv2.imshow("Recording", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
out.release()
cv2.destroyAllWindows()
```

Example (Roman Urdu): Aap apni webcam se 10 second ka clip record kar ke `output.avi` mein save kar sakte hain, bilkul mobile ke video recorder ki tarah.

EN: Common codecs: XVID with .avi, mp4v with .mp4.

> **Roman Urdu: Aam codecs:** .avi ke liye XVID aur .mp4 ke liye mp4v.

Module 7: Image Filtering and Edge Detection

7.1 Filtering (blurring)

EN: Filtering slides a small window (**kernel**) over the image and replaces each pixel by a combination of its neighbours. Blurring removes **noise** (random tiny specks).

> **Roman Urdu:** Filtering mein ek chhoti window (**kernel**) image par chalti hai aur har pixel ko uske aas paas ke pixels ke hisaab se badalti hai. Blur **noise** (bekaar chhote dhabbe) kam karta hai.

```
blur_avg   = cv2.blur(img, (5, 5))              # simple average
blur_gauss = cv2.GaussianBlur(img, (5, 5), 0)   # weighted, natural look
blur_med   = cv2.medianBlur(img, 5)             # best for salt-and-pepper noise
blur_bi    = cv2.bilateralFilter(img, 9, 75, 75) # blur but keep edges sharp
```

> **Roman Urdu:** Kernel size **hamesha taaq (odd)** number hota hai: 3, 5, 7. Bara kernel = zyada blur.

Example (Roman Urdu): Purani ghari ki photo par safed chhote dhabbe hain (salt-and-pepper noise). **medianBlur** unhein saaf kar dega. Chehre ko smooth karna ho lekin aankhon ki lakeerein na mitein to **bilateralFilter** use karein.

7.2 Edge detection (Canny)

EN: An **edge** is a place where brightness changes suddenly. Canny is the most popular detector. Steps: blur, then find gradients, then keep strong edges using two thresholds.

> **Roman Urdu: Edge** wo jagah hai jahan brightness achanak badalti hai (jaise cheez ki boundary). Canny sab se mashhoor detector hai. Pehle blur karte hain, phir do thresholds se mazboot edges rakhte hain.

```
gray  = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray  = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(gray, 100, 200)    # (lower, upper) thresholds
```

> **Roman Urdu: Lower threshold** se neeche wale edges reject, **upper** se upar wale pakke edges, beech wale tab rakhe jate hain jab wo kisi pakke edge se juray hon.

Example (Roman Urdu): Kisi gaari ki photo par Canny chalayein to sirf uski outline (bahar ki lakeerein) nazar aayegi aur baqi sab kala hoga, jaise pencil sketch.

Module 8: Image Features and Image Alignment

8.1 What are features?

EN: A **feature** (keypoint) is a distinctive spot in an image, such as a corner, that can be found again in another photo of the same scene. Each keypoint has a **descriptor**, a list of numbers that describes the area around it.

Roman Urdu: Feature (keypoint) image ka wo khaas point hai (jaise kona) jo usi scene ki doosri photo mein bhi dobara dhoondha ja sake. Har keypoint ka ek **descriptor** hota hai, yani numbers ki list jo uske aas paas ka hissa bayan karti hai.

Example (Roman Urdu): Do log ek hi imarat ki alag alag angle se photo lein. Imarat ke konay dono photos mein dhoondh kar match kiye ja sakte hain, bilkul jaise do jagah ek hi dost ko chehre se pehchan lena.

8.2 Detect and match with ORB

```
im1 = cv2.imread("form_scan.jpg", 0)     # photo we want to fix
im2 = cv2.imread("form_template.jpg", 0) # reference image

orb = cv2.ORB_create(5000)
kp1, des1 = orb.detectAndCompute(im1, None)
kp2, des2 = orb.detectAndCompute(im2, None)

matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(matcher.match(des1, des2), key=lambda m: m.distance)
good = matches[:int(len(matches) * 0.15)]    # keep best 15 percent
```

Roman Urdu: ORB ek tez aur free feature detector hai. **BFMatcher** dono images ke descriptors ko ek doosre se milata hai. **distance** jitni kam, match utna behtar. Hum sirf behtareen matches rakhte hain.

8.3 Image alignment with Homography

EN: A **homography** is a 3x3 matrix that maps points from one image to another. Using matched points we compute it, then **warp** the image so it lines up with the reference.

Roman Urdu: Homography ek 3x3 matrix hai jo ek image ke points ko doosri image ke points par map karti hai. Matched points se hum ise nikalte hain aur phir image ko **warp** (tera medha) karke reference ke mutabiq seedha kar dete hain.

```
import numpy as np
pts1 = np.float32([kp1[m.queryIdx].pt for m in good])
pts2 = np.float32([kp2[m.trainIdx].pt for m in good])

H, mask = cv2.findHomography(pts1, pts2, cv2.RANSAC)

h, w = im2.shape
aligned = cv2.warpPerspective(cv2.imread("form_scan.jpg"), H, (w, h))
```

Roman Urdu: RANSAC ghalat matches (outliers) ko ignore kar ke sahi homography dhoondta hai.

Example (Roman Urdu): Aapne form ki photo tirchi khinchi. Template se match karke homography nikalte hain aur **warpPerspective** us photo ko bilkul seedha scanner jaisa bana deta hai. Isi se document scanner apps chalti hain. Isi tarah panorama (kai photos jorna) bhi bante hain.

Quick Revision Cheat Sheet

Function	Kaam (Roman Urdu)
`cv2.imread / imshow / imwrite`	Image parho / dikhao / save karo
`cv2.cvtColor`	Colour space badlo (BGR to RGB/GRAY/HSV)
`cv2.split / merge`	Channels alag / wapis jor do
`img[y1:y2, x1:x2]`	Crop karo
`cv2.resize`	Size badlo (width, height)
`cv2.flip`	Ulta / mirror karo
`line, circle, rectangle, putText`	Shapes aur text banao
`cv2.add / subtract / multiply`	Brightness aur contrast
`cv2.threshold / adaptiveThreshold`	Kala safed (binary) image
`cv2.bitwise_and/or/xor/not`	Mask ke saath hissa rakho / hatao
`cv2.VideoCapture`	Camera ya video file kholo
`cv2.VideoWriter`	Video save karo
`GaussianBlur / medianBlur`	Noise kam karo
`cv2.Canny`	Edges nikalo
`ORB + BFMatcher`	Features dhoondo aur match karo
`findHomography + warpPerspective`	Image ko align karo

Common mistakes (Aam ghaltiyan)

- Forgetting BGR vs RGB. BGR aur RGB ka farq bhool jana, rang ulta aa jata hai.
- Mixing (width, height) and shape (height, width). resize aur shape ke order mein ghalti.
- Not calling cap.release(). cap.release() na likhna.
- Even kernel size in blur. Blur mein kernel size taaq (odd) na rakhna.